MVC Pattern in the CAP Flight & Booking Project

Freestyle SAPUI5 on CAP OData V4 — SAP Fiori Training

CAP itself is not an MVC framework — it's a Model-and-Service backend (db/schema.cds + srv/*.cds + srv/*.js from the previous document). MVC as a pattern shows up on the SAPUI5 side that consumes the CAP OData V4 service. There are two ways to build that frontend, and they map onto MVC very differently:

- Freestyle SAPUI5 — you write the View (XML) and Controller (JS) yourself; classic textbook MVC
- Fiori Elements — the View is generated at runtime from UI annotations, and the Controller is mostly framework code you only extend when needed

This document builds the freestyle version against the same Flights/Bookings CAP service, then contrasts it with Fiori Elements at the end.

1. Where each MVC layer sits

Layer	Freestyle SAPUI5	Fiori Elements
Model	sap.ui.model.odata.v4.ODataModel, configured once in manifest.json, bound to /odata/v4/flight/	Same OData V4 model — wired automatically by the FE template, no manifest changes beyond the data source
View	XML views you write: Flights.view.xml, Booking.fragment.xml	Generated at runtime from srv/flight-service-ui.cds — no view files exist in the project
Controller	Flights.controller.js — you write onInit, CRUD handlers, action calls	Controller extensions (extensionAPI) — optional hooks into the generated controller's lifecycle
Business logic	Stays server-side in srv/flight-service.js — never duplicated in the UI controller	Identical — same srv/flight-service.js, untouched

2. Combined project structure

```
flight-demo/
├── db/schema.cds                    <- Model (data): Flights + Bookings
├── srv/flight-service.cds            <- Service definition + actions
├── srv/flight-service-ui.cds         <- Fiori Elements annotations (not used by freestyle app)
├── srv/flight-service.js             <- validation + action handlers
└── app/flights/webapp/                <- freestyle SAPUI5 MVC app
    ├── Component.js
    ├── manifest.json                  <- Model (UI): OData V4 model config
    ├── view/
    │   ├── App.view.xml
    │   ├── Flights.view.xml           <- View
    │   └── Booking.fragment.xml        <- View (nested table)
    ├── controller/
    │   ├── App.controller.js
    │   └── Flights.controller.js      <- Controller
    └── i18n/i18n.properties
```

Everything above db/ and srv/ is unchanged from the CAP Flight/Booking service you already have. The MVC layer is entirely new code added under app/flights/webapp/ — CAP doesn't need to know whether a freestyle app or Fiori Elements sits on top of it.

3. Model — manifest.json OData V4 model configuration

```
"sap.app": {
  "id": "sap.fe.demo.flight.app",
  "dataSources": {
    "mainService": {
      "uri": "/odata/v4/flight/",
      "type": "OData",
      "settings": { "odataVersion": "4.0" }
    }
  }
},

"sap.ui5": {
  "models": {
    "": {
      "dataSource": "mainService"
    }
  },
  "dependencies": {
    "minUI5Version": "1.120.0",
    "libs": { "sap.ui.core": {}, "sap.m": {}, "sap.f": {} }
  }
}
```

This is the only place the OData V4 model is defined. Every view in the app binds against it declaratively (with {/Flights} paths) — the controller never manually creates the model.

4. View — Flights.view.xml

```
<mvc:View
  controllerName="sap.fe.demo.flight.app.controller.Flights"
  xmlns:mvc="sap.ui.core.mvc"
  xmlns="sap.m"
  xmlns:f="sap.f">

  <Page title="Flights">
    <content>
      <Table id="flightsTable"
             items="{/Flights}"
             mode="SingleSelectMaster"
             selectionChange="onFlightSelected">
        <columns>
          <Column><Text text="Carrier"/></Column>
          <Column><Text text="Connection"/></Column>
          <Column><Text text="Date"/></Column>
          <Column><Text text="Price"/></Column>
          <Column><Text text="Seats"/></Column>
        </columns>
        <items>
          <ColumnListItem>
            <cells>
              <Text text="{carrierID}"/>
              <Text text="{connectionID}"/>
              <Text text="{flightDate}"/>
              <ObjectStatus text="{price}"
                state="{= ${priceCriticality} === 3 ? 'Success'
                        : ${priceCriticality} === 2 ? 'Warning' : 'Error' }"/>
              <Text text="{seatsOcc}/{seatsMax}"/>
            </cells>
          </ColumnListItem>
        </items>
      </Table>
    </content>
```

```
    <footer>
      <Toolbar>
        <ToolbarSpacer/>
        <Button text="New Flight" press="onCreateFlight"/>
        <Button text="Delete" press="onDeleteFlight"/>
        <Button text="Cancel Flight" press="onCancelFlight"/>
        <Button text="Open Booking Window" press="onOpenBookingWindow"/>
      </Toolbar>
    </footer>
  </Page>
</mvc:View>
```

Same fields as the CAP model from the previous document: carrierID, connectionID, price, seatsMax/seatsOcc, and priceCriticality driving the ObjectStatus color — the UI-side equivalent of the @UI.LineItem Criticality annotation Fiori Elements would apply automatically.

5. View — Booking.fragment.xml (nested Bookings table)

```
<core:FragmentDefinition
  xmlns:core="sap.ui.core"
  xmlns="sap.m">
  <Table id="bookingsTable"
         items="{bookings}"
         mode="Delete"
         delete="onDeleteBooking">
    <columns>
      <Column><Text text="Booking ID"/></Column>
      <Column><Text text="Customer"/></Column>
      <Column><Text text="Passenger"/></Column>
      <Column><Text text="Class"/></Column>
      <Column><Text text="Amount"/></Column>
    </columns>
    <items>
      <ColumnListItem>
        <cells>
          <Text text="{bookingID}"/>
          <Text text="{customerID}"/>
          <Text text="{passengerName}"/>
          <Text text="{bookingClass}"/>
          <Text text="{foreignCurrencyAmount}"/>
        </cells>
      </ColumnListItem>
    </items>
  </Table>
</core:FragmentDefinition>
```

items="{bookings}" is a relative binding — it's inserted into the view already bound to a selected Flight's context (step 6), so it reads that Flight's own bookings composition, same navigation property from the CAP model.

6. Controller — Flights.controller.js

```
sap.ui.define([
  "sap/ui/core/mvc/Controller",
  "sap/m/MessageToast",
  "sap/m/MessageBox"
], function (Controller, MessageToast, MessageBox) {
  "use strict";

  return Controller.extend("sap.fe.demo.flight.app.controller.Flights", {

    onInit: function () {
      // The model is already bound declaratively in the view ({/Flights});
      // nothing to wire up manually for OData V4.
    },
```

```
    onFlightSelected: function (oEvent) {
      this._oSelectedFlight = oEvent.getParameter("listItem").getBindingContext();
      // e.g. bind the Booking.fragment.xml table to this._oSelectedFlight + "bookings"
    },

    onCreateFlight: function () {
      var oListBinding = this.byId("flightsTable").getBinding("items");
      var oContext = oListBinding.create({
        carrierID: "LH",
        connectionID: "0400",
        flightDate: "2026-10-01",
        price: 450.00,
        currencyCode: "USD"
      });
      oContext.created().then(function () {
        MessageToast.show("Flight created");
      }, function (oError) {
        // req.error() thrown by the CAP validation handler surfaces here
        MessageBox.error(oError.message);
      });
    },

    onDeleteFlight: function () {
      if (this._oSelectedFlight) {
        this._oSelectedFlight.delete();
      }
    },

    onCancelFlight: function () {
      if (!this._oSelectedFlight) { return; }
      var oOperation = this._oSelectedFlight.getModel().bindContext(
        "FlightService.cancelFlight(...)", this._oSelectedFlight
      );
      oOperation.execute().then(function () {
        MessageToast.show("Flight cancelled");
      });
    },

    onOpenBookingWindow: function () {
      var oModel = this.getView().getModel();
      var oOperation = oModel.bindContext("/openBookingWindow(...)");
      oOperation.setParameter("forDate", "2026-10-01");
      oOperation.execute()
        .then(function () { return oOperation.getBoundContext().requestObject(); })
        .then(function (oResult) { MessageToast.show(oResult.value); });
    },

    onDeleteBooking: function (oEvent) {
      oEvent.getParameter("listItem").getBindingContext().delete();
    }
  });
});
```

Notice what's absent: no SQL, no seat-count check, no business rule. The Controller only orchestrates OData V4 calls (create, delete, bindContext for actions) — it's the same separation of concerns UX400 teaches for any SAPUI5 app, just now sitting in front of CAP instead of a Gateway/RAP service.

7. The Fiori Elements alternative — no hand-written View or Controller

Point this service at sap.fe.templates.ListReport / sap.fe.templates.ObjectPage instead of a freestyle app (the same fiori add generator flow from earlier sessions), and steps 4-6 disappear entirely. The View is generated from srv/flight-service-ui.cds; the Controller is framework code. You only add a file when you need to hook into it:

```
// webapp/ext/controller/FlightsExt.js
sap.ui.define(["sap/fe/core/PageController"], function (PageController) {
  "use strict";
  return PageController.extend("sap.fe.demo.flight.app.ext.FlightsExt", {
    onBeforeCreate: function () {
      // optional hook into the FE-generated controller's lifecycle
    }
  });
});
```

```
// manifest.json
"extends": {
  "extensions": {
    "sap.ui.controllerExtensions": {
      "sap.fe.templates.ListReport.ListReportController": {
        "controllerName": "sap.fe.demo.flight.app.ext.FlightsExt"
      }
    }
  }
}
```

This is the same smart-controls-are-deprecated point from Session 3: with Fiori Elements, you reach for a controller extension only when an annotation genuinely can't express what you need — same rule as reaching for FPM custom sections.

8. Where validation lives — one source of truth either way

Both flavors call the same CAP service, so the seatsOcc-vs-seatsMax check from srv/flight-service.js runs identically whether the request came from Flights.controller.js above or from the Fiori Elements generated controller. The UI layer — freestyle or FE — never re-implements that rule; it only displays the error CAP sends back:

```
// what the CAP handler sends:
req.error(400, 'Booked seats cannot exceed planned seats');

// what the freestyle controller does with it (see onCreateFlight above):
MessageBox.error(oError.message);

// what Fiori Elements does with it: shows the same message
// automatically in its built-in message popover — no controller code needed
```

That's the practical takeaway for the demo: Model and business logic (db/schema.cds, srv/flight-service.js) stay identical no matter which MVC flavor sits on top; only the View and Controller layer changes.